

UD9 - Monitorización, análisis y optimización del rendimiento en sistemas

9.1 Introducción y objetivos de la unidad

9.1.1 Resultado de aprendizaje

RA6: Detecta problemas de rendimiento, monitorizando el sistema con las herramientas adecuadas y documentando el procedimiento.

Criterios de evaluación:

a) Se han identificado los objetos monitorizables en un sistema informático.

b) Se han identificado los tipos de sucesos.

c) Se han utilizado herramientas de monitorización en tiempo real.

d) Se ha monitorizado el rendimiento mediante registros de contador y de seguimiento del sistema.

e) Se han planificado y configurado alertas de rendimiento.

f) Se han interpretado los registros de rendimiento almacenados.

g) Se ha analizado el sistema mediante técnicas de simulación para optimizar el rendimiento.

h) Se ha elaborado documentación de soporte y de incidencias.

📎 Fuente: BOE-A-2009-18355, Anexo I, Módulo Profesional 0369 – Implantación de sistemas operativos.

¿Relación con el módulo 0371 FH?

RA1: Configura equipos microinformáticos, componentes y periféricos, analizando sus características y relación con el conjunto.

Criterios de evaluación:

e) Se ha evaluado las prestaciones del equipo.

f) Se han ejecutado utilidades de chequeo y diagnóstico.

g) Se han identificado averías y sus causas.

📎 Fuente: BOE-A-2009-18355, Anexo I, Módulo Profesional 0369 – Implantación de sistemas operativos.

9.1.2 Estructura de la unidad

Esta unidad se organiza en los siguientes apartados:

9.1 Introducción y objetivos de la unidad
9.2 Fundamentos de la monitorización del sistema (RA6.a, RA6.b)
9.3 Herramientas de monitorización y recogida de datos (RA6.c, RA6.d, RA6.e)
9.4 Análisis e interpretación del rendimiento (RA6.f)
9.5 Análisis del sistema mediante simulación y optimización (RA6.g, RA6.h)

8.1.2 Estructura de la unidad (índice detallado)

9.1 Introducción y objetivos de la unidad
9.1.1 Resultado de aprendizaje
9.1.2 Estructura de la unidad
9.1.3 Actividades previstas
9.1.4 Recursos y entorno de trabajo

9.2 Fundamentos de la monitorización del sistema (RA6.a, RA6.b)
9.2.1 Concepto de monitorización del sistema
9.2.2 Objetos monitorizables en un sistema informático
9.2.3 Tipos de sucesos del sistema
9.2.4 Relación entre eventos y rendimiento del sistema

9.3 Herramientas de monitorización y recogida de datos (RA6.c, RA6.d, RA6.e)
9.3.1 Monitorización en tiempo real
9.3.2 Monitorización mediante registros del sistema
9.3.3 Métricas y monitorización del sistema
9.3.4 Configuración de alertas de rendimiento

8.1.2 Estructura de la unidad (índice detallado)

9.4 Análisis e interpretación del rendimiento (RA6.f)

9.4.1 Interpretación de métricas de rendimiento
9.4.2 Estados de los procesos
9.4.3 Identificación de problemas de rendimiento
9.4.4 Reglas básicas para el análisis del rendimiento

9.5 Análisis del sistema mediante simulación y optimización (RA6.g, RA6.h)

9.5.1 Concepto de simulación de carga
9.5.2 Herramientas de simulación de carga
9.5.3 Herramientas de evaluación del rendimiento
9.5.4 Ejercicio práctico: simulación y optimización del sistema
9.5.5 Documentación y gestión de incidencias

9.1.3 Actividades previstas

Durante el desarrollo de esta unidad se realizarán las siguientes actividades:

En un primer bloque, se abordarán en el aula los conceptos fundamentales relacionados con la monitorización, el análisis y la optimización del rendimiento en sistemas informáticos.
Se explicarán las métricas básicas del sistema, los tipos de sucesos y las herramientas de monitorización en entornos GNU/Linux.
Asimismo, se realizarán **ejercicios prácticos guiados** orientados al manejo de herramientas como top, htop o journalctl, así como a la interpretación de los datos obtenidos y a la aplicación de técnicas básicas de optimización.

En un segundo bloque, estos conocimientos se aplicarán en el contexto del **reto intermodular**, donde el alumnado deberá monitorizar, analizar y optimizar el rendimiento de las infraestructuras desplegadas. Dado que el entorno de trabajo se basa en **equipos reutilizados y con recursos limitados** (memoria, CPU y almacenamiento), y en el uso de **software libre**, la monitorización continua del sistema será un elemento clave para detectar las limitaciones reales del entorno.

Este enfoque requiere una **monitorización continua del sistema**, imprescindible para detectar problemas, tomar decisiones y aplicar mejoras que aseguren su correcto funcionamiento.

9.1.4 Recursos y entorno de trabajo

Sistemas operativos de práctica:

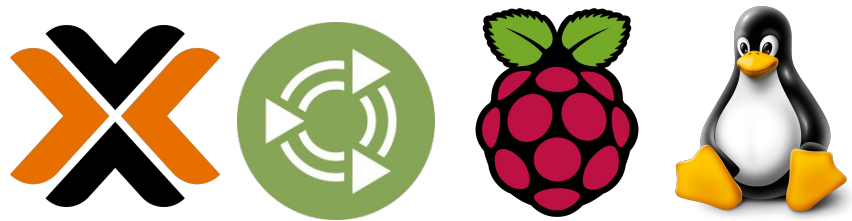
- Sistemas operativos de servidor y cliente seleccionados por el alumnado en función de los requisitos del proyecto.
- Elección del sistema operativo en función del hardware disponible y de la funcionalidad a implementar (servidores de archivos, web, controladores de dominio, clientes, etc.).

Entorno de trabajo:

- Infraestructura basada en **mini PCs, equipos torre no actuales y dispositivos como Raspberry Pi**.
- Equipos con **recursos limitados**, que condicionan la elección del sistema operativo y la configuración de los servicios.
- Entornos de **virtualización** que permitan la ejecución de varios sistemas sobre un mismo equipo físico.
- Despliegue de servidores y clientes en red según el diseño del reto.

Material de apoyo:

- Documentación técnica y guías sobre monitorización y análisis del sistema.
- Recursos y materiales facilitados en el aula virtual.



9.2 Fundamentos de la monitorización del sistema



En este apartado se introducen los fundamentos de la **monitorización del sistema**, identificando los elementos que pueden ser observados y los tipos de sucesos que se generan durante su funcionamiento.
Se analizará cómo estos indicadores permiten obtener una visión general del estado del sistema y sirven como base para la detección de posibles problemas de rendimiento.
Asimismo, se establecerá la relación entre los eventos del sistema y el uso de los recursos, sentando las bases para su posterior análisis y optimización.

9.2.1 Concepto de monitorización del sistema

La **monitorización del sistema** es el proceso de recopilación, medición y análisis de datos relativos al uso de los recursos y al comportamiento de un sistema informático.

Se basa en la obtención de **métricas de rendimiento** (CPU, memoria, E/S, red) y en el registro de **eventos del sistema**, que permiten evaluar su estado operativo.

A partir de estos datos, es posible **detectar desviaciones respecto a un funcionamiento esperado**, identificar cuellos de botella y aplicar acciones de optimización.

En entornos de administración, la monitorización se integra como un proceso continuo de **supervisión, análisis y mejora del sistema**.

9.2.2 Objetos monitorizables en un sistema informático

La monitorización de un sistema se basa en el análisis de los principales recursos que intervienen en su funcionamiento.

Estos recursos pueden agruparse en los siguientes elementos:

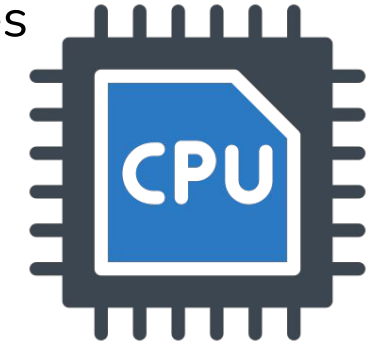
- procesador (CPU),
- memoria,
- almacenamiento,
- red,
- procesos y servicios,
- y sistema global.

El estudio de cada uno de estos componentes permite identificar el origen de los problemas de rendimiento y evaluar el estado general del sistema.

9.2.2 Objetos monitorizables en un S.I. (I)

Procesador (CPU)

El procesador es el recurso encargado de ejecutar las instrucciones del sistema y de los procesos en ejecución. Su monitorización permite evaluar la carga de trabajo y detectar posibles situaciones de saturación.



Se analizan los siguientes indicadores:

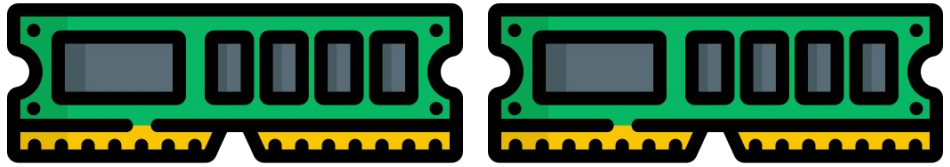
- **Uso de CPU (%):** porcentaje de tiempo en el que la CPU está ocupada.
- **Modo usuario / modo sistema:** tiempo dedicado a procesos de usuario y al núcleo del sistema.
- **Tiempo de espera (I/O wait):** tiempo en el que la CPU permanece inactiva esperando operaciones de entrada/salida.
- **Interrupciones y cambios de contexto:** reflejan la actividad interna del sistema y la gestión de procesos.

Un uso elevado y sostenido de CPU puede indicar una sobrecarga del sistema o procesos mal optimizados. Por otro lado, valores altos de espera (I/O wait) pueden evidenciar cuellos de botella en el subsistema de almacenamiento o red.

9.2.2 Objetos monitorizables en un S.I. (II)

Memoria principal (RAM)

La memoria principal es el recurso encargado de almacenar temporalmente los datos y procesos en ejecución. Su monitorización permite evaluar la capacidad del sistema para gestionar la carga de trabajo sin recurrir a mecanismos más lentos como el almacenamiento secundario.



Se analizan los siguientes indicadores:

- **Memoria utilizada, libre y disponible:** reflejan el estado general del uso de la RAM.
- **Buffers y caché:** memoria utilizada por el sistema para optimizar accesos a disco.
- **Uso de swap:** espacio en disco utilizado cuando la memoria RAM es insuficiente.

Un uso elevado de memoria no implica necesariamente un problema, ya que el sistema operativo tiende a aprovechar la RAM disponible mediante caché.

Sin embargo, el uso intensivo de **swap** es un indicador crítico de presión de memoria, ya que implica acceso a disco y una degradación significativa del rendimiento.

9.2.2 Objetos monitorizables en un S.I. (III)

Almacenamiento (E/S de disco)

El subsistema de almacenamiento es responsable de las operaciones de entrada/salida (E/S), fundamentales para el acceso a datos y el funcionamiento de aplicaciones y servicios.

Su monitorización permite detectar cuellos de botella que pueden limitar el rendimiento global del sistema.



Se analizan los siguientes indicadores:

- **Tasa de lectura/escritura:** cantidad de datos transferidos por unidad de tiempo.
- **Latencia:** tiempo de respuesta en el acceso a los datos.
- **Operaciones de E/S por segundo (IOPS):** número de accesos al dispositivo.
- **Porcentaje de uso del dispositivo:** tiempo durante el cual el disco está ocupado.

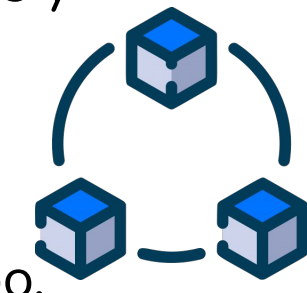
Un sistema puede presentar bajo uso de CPU y, sin embargo, tener un rendimiento deficiente debido a un subsistema de almacenamiento saturado.

Altos tiempos de latencia o un uso cercano al 100% indican un posible cuello de botella en disco.

9.2.2 Objetos monitorizables en un S.I. (IV)

Red

El subsistema de red gestiona la comunicación entre sistemas y el acceso a servicios distribuidos. Su monitorización permite evaluar el rendimiento de las comunicaciones y detectar limitaciones en la transmisión de datos.



Se analizan los siguientes indicadores:

- **Tráfico entrante y saliente:** volumen de datos transmitidos por unidad de tiempo.
- **Uso del ancho de banda:** grado de ocupación respecto a la capacidad disponible.
- **Latencia:** tiempo de respuesta en la comunicación entre sistemas.
- **Pérdida de paquetes:** datos que no llegan a su destino durante la transmisión.
- **Errores de transmisión:** fallos detectados en la interfaz de red.

La saturación del enlace, el aumento de la latencia o la pérdida de paquetes pueden degradar significativamente el rendimiento de los servicios, especialmente en entornos distribuidos.

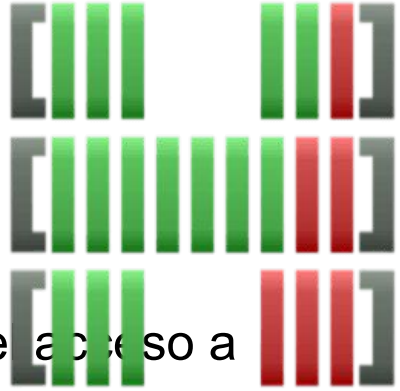
9.2.2 Objetos monitorizables en un S.I. (V)

Procesos y servicios

Los procesos representan las unidades de ejecución del sistema, mientras que los servicios (daemons) son procesos que se ejecutan en segundo plano proporcionando funcionalidades específicas.

La monitorización de procesos y servicios permite identificar el consumo de recursos a nivel individual y detectar comportamientos anómalos. Se analizan los siguientes indicadores:

- **Consumo de CPU y memoria por proceso:** identifica qué procesos están utilizando más recursos.
- **Estado del proceso:** ejecución, espera, bloqueo o procesos zombie.
- **Número de procesos activos:** cantidad total y distribución de carga.
- **Disponibilidad de servicios:** funcionamiento correcto de los servicios del sistema.



Un proceso puede monopolizar recursos del sistema y degradar el rendimiento global. Asimismo, procesos bloqueados o en estados anómalos pueden indicar problemas en el acceso a recursos como disco o red.

La monitorización de servicios es fundamental para garantizar la disponibilidad de los sistemas, ya que su fallo puede afectar directamente al funcionamiento de aplicaciones y usuarios.

9.2.2 Objetos monitorizables en un S.I. (VI)

Sistema global

Además de los recursos individuales, es necesario monitorizar indicadores que reflejen el estado general del sistema, proporcionando una visión agregada de su comportamiento.

Se analizan los siguientes indicadores:

- **Carga del sistema (load average):** número medio de procesos en ejecución o en espera de CPU.
- **Tiempo de actividad (uptime):** tiempo transcurrido desde el último arranque del sistema.
- **Número de usuarios conectados:** sesiones activas en el sistema.

La **carga del sistema** es uno de los indicadores más relevantes, ya que permite evaluar si el sistema está sobredimensionado o saturado en función del número de procesadores disponibles.

Un valor de carga elevado de forma sostenida indica que el sistema no puede atender todas las tareas de forma eficiente.

Estos indicadores permiten obtener una visión global del estado del sistema y sirven como punto de partida para un análisis más detallado de los recursos individuales.

9.2.3 Tipos de sucesos del sistema

El funcionamiento de un sistema informático genera de forma continua **sucesos o eventos**, que son registros de actividades relevantes producidas por el sistema operativo, los servicios y las aplicaciones.

Estos sucesos permiten analizar el comportamiento del sistema y detectar incidencias o situaciones anómalas.

Se clasifican principalmente en los siguientes tipos:

- **Eventos informativos:**
  Indican el funcionamiento normal del sistema o de un servicio. No implican ningún problema.
- **Eventos de advertencia:**
  Señalan situaciones que pueden derivar en un problema si persisten, como uso elevado de recursos o fallos no críticos.
- **Eventos de error:**
  Indican fallos en el sistema o en algún servicio que afectan a su funcionamiento.

9.2.4 Relación entre eventos y rendimiento del sistema

El rendimiento de un sistema informático no depende únicamente del uso de sus recursos, sino también de los **eventos que se producen durante su funcionamiento**.

Los sucesos registrados en el sistema permiten **contextualizar las métricas de rendimiento**, facilitando la identificación del origen de posibles problemas.

Por ejemplo:

- un aumento en el uso de CPU puede estar relacionado con el inicio de un servicio,
- errores en disco pueden provocar tiempos de espera elevados,
- fallos en red pueden generar interrupciones en servicios.

El análisis conjunto de **métricas (CPU, memoria, E/S, red)** y **eventos del sistema** permite establecer relaciones causa-efecto y realizar un diagnóstico más preciso.

De este modo, los eventos no solo informan de incidencias, sino que ayudan a explicar el comportamiento del sistema y a orientar el proceso de análisis.

9.3 Herramientas de monitorización y recogida de datos



La monitorización de un sistema no solo implica conocer qué recursos se están utilizando, sino también disponer de herramientas que permitan observar, registrar y analizar su comportamiento en tiempo real y a lo largo del tiempo. En este apartado se abordarán las principales herramientas y mecanismos utilizados para recoger datos del sistema, sentando las bases para su posterior análisis y optimización.

9.3.1 Monitorización en tiempo real (I)

La monitorización en tiempo real permite observar el estado del sistema de forma instantánea, mostrando el uso de los recursos mientras los procesos están en ejecución. Este tipo de herramientas es fundamental para detectar comportamientos anómalos **en el momento en que se producen**.

En entornos GNU/Linux, se utilizan herramientas que muestran información dinámica sobre CPU, memoria, procesos y carga del sistema directamente desde la terminal.

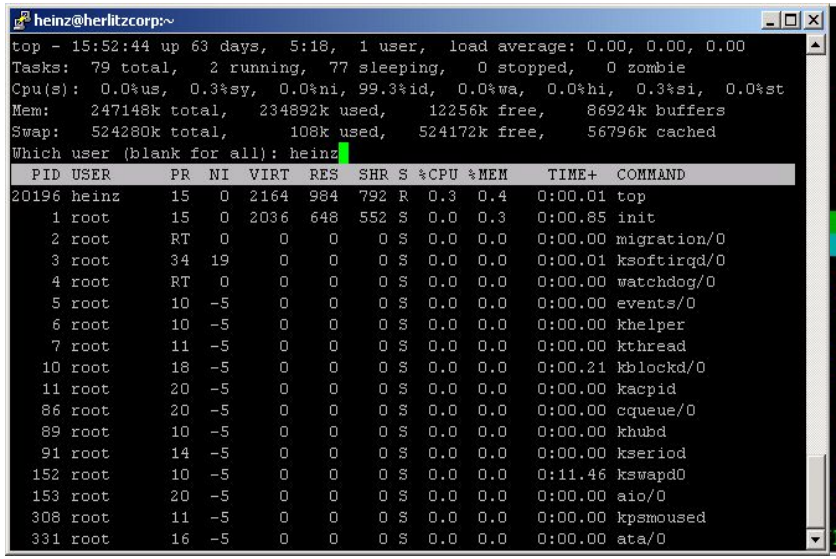
Principales herramientas:

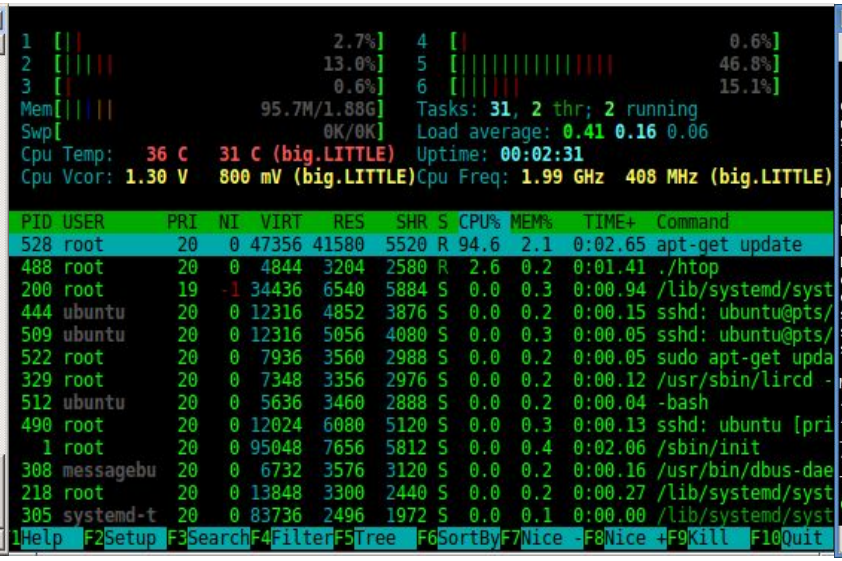
- top: muestra procesos activos y uso de recursos en tiempo real.
- htop: versión mejorada e interactiva de top, con visualización más clara.
- glances: herramienta avanzada que integra múltiples métricas del sistema en una sola vista.

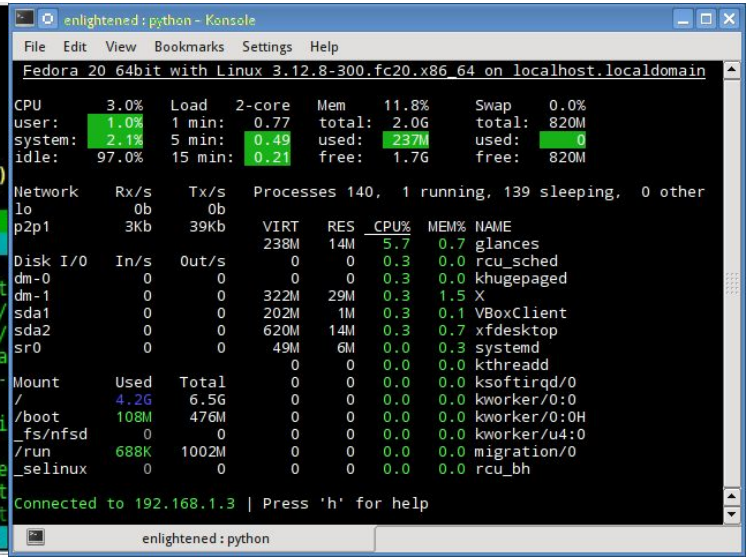
Estas herramientas permiten al administrador obtener una visión inmediata del comportamiento del sistema y constituyen el primer paso en el proceso de monitorización.

9.3.1 Monitorización en tiempo real (II)

Herramienta	Interfaz	Interactividad	Información	Instalación	Uso recomendado
top	Texto básico	Baja	Procesos y CPU	Por defecto	Diagnóstico rápido
htop	Visual (colores)	Alta (teclado + ratón)	CPU, RAM, procesos	Manual	Uso diario
glances	Dashboard terminal	Media	Sistema completo	Manual	Visión global







9.3.2 Monitorización mediante registros del sistema (I)

Los sistemas operativos registran de forma continua eventos relacionados con su funcionamiento, permitiendo su consulta posterior para analizar el comportamiento del sistema a lo largo del tiempo. Esta monitorización no es en tiempo real, sino basada en el almacenamiento de información en archivos de registro.

En GNU/Linux, los registros del sistema se almacenan en archivos accesibles desde el sistema de ficheros. Estos son sus elementos clave:

Ubicación principal: /var/log/

Tipos de registros:

- **Sistema** → funcionamiento general
  - Ej: /var/log/syslog, /var/log/kern.log
- **Seguridad** → autenticación y accesos
  - Ej: /var/log/auth.log
- **Servicios** → eventos de aplicaciones concretas
  - Ej: /var/log/apache2/, /var/log/mysql/, /var/log/cron.log

Consulta de registros: Herramientas como journalctl

9.3.2 Monitorización mediante registros del sistema (II)

Comparativa de registros: Linux vs Windows

Linux

- Archivos en /var/log/
- Formato texto plano
- Acceso directo desde terminal
- Ej: syslog, auth.log

Windows

- Visor de eventos
- Registros estructurados
- Categorías principales:
  - Sistema
  - Seguridad
  - Aplicación

Idea clave:

Ambos sistemas operativos registran eventos del sistema, pero difieren en su **forma de almacenamiento y acceso**.

9.3.3 Métricas y monitorización del sistema (I)

Una **métrica** es un valor numérico que representa el estado o comportamiento de un recurso del sistema en un momento determinado.

Principales métricas:

- **CPU**
  - Uso (%) → porcentaje de tiempo que el procesador está trabajando
- **Memoria**
  - Uso → cantidad de memoria en uso
  - Disponible → memoria libre o reutilizable
- **Disco**
  - Uso → espacio ocupado en almacenamiento
  - Lectura/Escritura (I/O) → actividad de acceso al disco (datos que se leen o escriben)
- **Sistema**
  - Procesos → número de procesos activos
  - Carga media → nivel de trabajo del sistema en el tiempo

Estas métricas permiten describir de forma cuantitativa el comportamiento de los recursos del sistema.

9.3.3 Métricas y monitorización del sistema (II)

Herramientas de monitorización de métricas

En GNU/Linux, existen herramientas específicas que permiten obtener y consultar las métricas del sistema de forma detallada.

Herramientas habituales:

- vmstat → estado de memoria, procesos y actividad del sistema
- iostat → actividad de entrada/salida en dispositivos de almacenamiento
- mpstat → uso de CPU por núcleo
- free → uso y disponibilidad de memoria
- uptime → carga media del sistema

Estas herramientas permiten obtener métricas específicas del sistema, a diferencia de herramientas de monitorización en tiempo real como top, htop o glances, que ofrecen una visión global e interactiva.

9.3.4 Configuración de alertas de rendimiento

La monitorización de un sistema puede complementarse con mecanismos que permiten detectar automáticamente situaciones anómalas sin necesidad de supervisión constante. Las alertas de rendimiento se basan en la definición de condiciones que, al cumplirse, generan una notificación o acción.

En GNU/Linux, estas alertas pueden implementarse mediante herramientas del sistema y automatización de tareas.

Elementos clave:

- Definición de umbrales (CPU, memoria, disco...)
- Detección automática de condiciones
- Generación de avisos (mensajes, logs, correo)
- Automatización mediante scripts y tareas programadas (cron)

Estas configuraciones permiten una gestión proactiva del sistema, facilitando la detección temprana de problemas.

9.3.4 (ampliación) – Automatización de tareas con cron

cron es un servicio de GNU/Linux que permite ejecutar comandos o scripts de forma automática en momentos programados.

Características:

- Permite ejecutar tareas periódicas
- Basado en tiempo (minutos, horas, días...)
- Muy utilizado para tareas de mantenimiento y monitorización

Ubicación y edición:

- Configuración por usuario:
  - crontab -e → editar tareas programadas
- Listar tareas:
  - crontab -l
- Archivos del sistema:
  - /etc/crontab
  - /etc/cron.*

9.3.4 (ejemplo) – Alerta de uso de CPU con cron

Objetivo:
Detectar si la CPU supera un umbral y generar un aviso.

Script de comprobación (`/usr/local/bin/check_cpu.sh`):

```
#!/bin/bash

CPU=$(top -bn1 | grep "Cpu(s)" | awk '{print 100 - $8}')

if (( $(echo "$CPU > 80" | bc -l) )); then
   echo "$(date) ALERTA: CPU por encima del 80% ($CPU%)" >> /var/log/alertas.log
fi
```

Tarea programada con `cron`:

```
*/5 * * * * /usr/local/bin/check_cpu.sh
```

Ejecuta el script cada 5 minutos.

Qué ocurre:

- Se comprueba el uso de CPU
- Si supera el umbral → se genera una alerta en log
- Permite detección automática sin supervisión

UD9.TrA - Herramientas de monitorización del sistema



UD9.TrA - Herramientas de monitorización del sistema (1/12)

Bloque 1 · Reconocimiento del sistema

Micropíldora

Antes de monitorizar, es necesario identificar el sistema: usuario, equipo y estado general.
Estos datos permiten contextualizar la información que veremos después.

Ejecuta:

```
whoami
hostname
uname -a
uptime
```

Observa:

- Usuario actual
- Nombre del equipo
- Versión del sistema
- Tiempo encendido

UD9.TrA - Herramientas de monitorización del sistema (2/12)

Bloque 2 · Memoria del sistema

Micropíldora (corregida)

La memoria RAM almacena los programas en ejecución.

- **total**: memoria instalada
- **used**: memoria en uso
- **free**: memoria sin uso
- **buff/cache**: memoria usada como caché (recuperable)
- **available**: memoria realmente disponible para nuevas aplicaciones
- **swap**: espacio en disco usado cuando la RAM se llena

Ejecuta:

```
free -h
```

Observa:

- Memoria total del sistema
- Memoria usada
- Memoria libre (free)
- Memoria disponible (available)
- Memoria en caché (buff/cache)
- Uso de swap (si aparece)

UD9.TrA - Herramientas de monitorización del sistema (3/12)

Bloque 3 · Monitorización en tiempo real (TOP)

Micropíldora

Un proceso es un programa en ejecución.

El sistema está formado por múltiples procesos que consumen CPU y memoria.

top permite observar esta actividad en tiempo real.

Ejecuta:

```
top
```

Observa:

- Uso de CPU (línea superior)
- Uso de memoria
- Lista de procesos activos
- Proceso con mayor consumo de CPU

UD9.TrA - Herramientas de monitorización del sistema (4/12)

Bloque 4 · Interacción con TOP

Acciones:

Dentro de top, prueba:

- P → ordenar por uso de CPU
- M → ordenar por uso de memoria
- k → finalizar un proceso (introduce su PID)
- h → mostrar ayuda

Comprueba:

- Cómo cambia el orden de los procesos
- Qué proceso aparece en primera posición
- Qué ocurre al finalizar un proceso

UD9.TrA - Herramientas de monitorización del sistema (5/12)

Bloque 5 · Monitorización mejorada (HTOP)

Ejecuta:

htop

Acciones:

- Usa F6 → ordenar procesos
- Usa F3 → buscar un proceso
- Usa F5 → vista en árbol (jerarquía de procesos)
- Usa F9 → finalizar un proceso

Compara con TOP:

- ¿Qué diferencias observas en la visualización?
- ¿Qué herramienta te resulta más clara?

UD9.TrA - Herramientas de monitorización del sistema (6/12)

Bloque 6 · Monitorización global (GLANCES)

Ejecuta:

```
sudo apt update
sudo apt install glances -y
glances
```

Observa:

- Uso de CPU
- Uso de memoria
- Actividad de disco
- Red
- Procesos

Fíjate en:

- El uso de colores (verde, amarillo, rojo), que indican el estado de cada recurso
- Qué recursos presentan mayor carga en cada momento (CPU, memoria, disco o red)
- Cómo toda la información del sistema se muestra de forma agrupada en una única pantalla

UD9.TrA - Herramientas de monitorización del sistema (7/12)

Bloque 7 · Generación de carga del sistema

Micropíldora

El comando yes genera un proceso que consume CPU de forma continua.
Se utiliza para simular carga en el sistema y observar su comportamiento.

Ejecuta:

```
yes > /dev/null &
yes > /dev/null &
```

Observa:

- Cómo aumenta el uso de CPU
- Qué procesos aparecen con mayor consumo
- Cómo cambia la información en top, htop o glances

Acciones:

- Localiza los procesos yes
- Finalízalos (desde top, htop o con kill)

Si el sistema se ralentiza, finaliza los procesos

UD9.TrA - Herramientas de monitorización del sistema (8/12)

Bloque 8 · Procesos (visión estática)

Micropíldora

Las herramientas como top o htop muestran procesos en tiempo real.

El comando ps permite obtener una "foto" del estado de los procesos en un momento concreto.

Ejecuta:

ps aux

ps aux | head

Observa:

- Lista de procesos en ejecución
- Usuario propietario de cada proceso
- Uso de CPU y memoria
- Diferencias respecto a top o htop

Acción:

ps aux | grep yes → Localiza los procesos yes que has creado anteriormente

UD9.TrA - Herramientas de monitorización del sistema (9/12)

Bloque 9 · Actividad del sistema (vmstat)

Ejecuta:

vmstat 1 5

Observa:

- Procesos en ejecución (r)
- CPU (us, sy, id)
- Memoria disponible
- Cambios entre cada actualización

Fíjate en:

- Cómo varían los valores en tiempo real
- Qué ocurre si el sistema tiene carga (puedes usar yes)
- Diferencias respecto a top o htop

UD9.TrA - Herramientas de monitorización del sistema (10/12)

Bloque 10 · Actividad de disco (iostat)

Ejecuta:

iostat

Si no está instalado:
sudo apt install sysstat -y

Observa:

- Dispositivos de disco
- Lecturas y escrituras
- Uso del disco (%util si aparece)

Fíjate en:

- Si hay actividad de entrada/salida
- Qué dispositivo tiene más carga
- Diferencias respecto a CPU y memoria

UD9.TrA - Herramientas de monitorización del sistema (11/12)

Bloque 11 · Uso de CPU por núcleo (mpstat)

Ejecuta:

mpstat -P ALL

Si no está instalado:
sudo apt install sysstat -y

Observa:

- Uso de CPU por cada núcleo
- Diferencias entre núcleos
- Valores de uso (us, sy, id)

Fíjate en:

- Si todos los núcleos trabajan igual - Identificar núcleos másа cargados
- Qué ocurre cuando el sistema tiene carga
- Relación con lo visto en top

UD9.TrA - Herramientas de monitorización del sistema (12/12)

Cierre · Síntesis del entrenamiento

Has trabajado con:

- Monitorización en tiempo real → top, htop, glances
- Comandos de análisis → ps, vmstat, iostat, mpstat
- Simulación de carga → yes

Has observado:

- Uso de CPU, memoria, disco y red
- Procesos en ejecución
- Comportamiento del sistema bajo carga

9.4 Análisis e interpretación del rendimiento



El análisis del rendimiento de un sistema informático va más allá de la simple obtención de métricas. Implica interpretar los datos recogidos, identificar patrones de comportamiento y establecer relaciones entre los distintos recursos del sistema. A partir de esta interpretación, es posible detectar anomalías, diagnosticar problemas y comprender el origen de las limitaciones que afectan al funcionamiento del sistema.

9.4.1 Interpretación de métricas de rendimiento (I)

Las métricas de rendimiento deben interpretarse como **indicadores relativos y dependientes del contexto**, no como valores absolutos. Su análisis requiere establecer un **comportamiento esperado del sistema (baseline)** y comparar los valores observados con dicho patrón. La detección de anomalías se basa en **desviaciones significativas respecto al comportamiento normal**, más que en la superación de umbrales fijos.

Además, el rendimiento de un sistema no depende de un único recurso, sino de la **interacción entre varios subsistemas**. Por ello, la interpretación correcta de las métricas exige analizar conjuntamente CPU, memoria, almacenamiento y procesos, identificando **relaciones causa-efecto**. Por ejemplo:

- Un sistema con baja utilización de CPU puede presentar bajo rendimiento debido a **esperas prolongadas de E/S**.
- Un uso intensivo de memoria acompañado de acceso a **swap** indica una degradación del rendimiento por limitación de recursos.

9.4.1 Interpretación de métricas de rendimiento (II)

Ejemplo de interpretación con baseline

- Sistema en condiciones normales:
  - CPU: 20–30%
  - Load average: < 1 por núcleo
  - I/O wait: bajo
- Sistema en análisis:
  - CPU: 25%
  - Load average: 4
  - I/O wait: elevado

Interpretación:

Aunque el uso de CPU es aparentemente normal, el aumento del load average junto con tiempos de espera elevados indica que los procesos están bloqueados esperando recursos, probablemente por un **cuello de botella en disco o E/S**.

9.4.2 Estados de los procesos (I)

Los procesos pueden encontrarse en distintos estados en función de su relación con la CPU y los recursos del sistema. Estos estados permiten interpretar cómo se distribuye la carga y detectar situaciones que afectan al rendimiento.

En sistemas Linux, herramientas como top, htop o ps permiten observar estos estados en tiempo real o de forma estática. Estos son los principales estados de los procesos:

- **Running (R)** Proceso en ejecución o listo para ejecutarse. Está utilizando CPU o en cola inmediata para hacerlo.
- **Sleeping (S)** Proceso en espera de algún evento o recurso (entrada/salida, temporizador, señal). Es el estado más habitual.
- **Uninterruptible sleep (D)** Proceso en espera de operaciones de E/S (disco, red). No puede ser interrumpido. Un número elevado puede indicar problemas de rendimiento.
- **Stopped (T)** Proceso detenido manualmente o por señal.
- **Zombie (Z)** Proceso finalizado que aún no ha sido recogido por su proceso padre.

9.4.2 Estados de los procesos (II)

Desde el punto de vista del rendimiento, los estados de los procesos permiten interpretar cómo se está utilizando la CPU y si existen cuellos de botella en el sistema. No todos los estados tienen el mismo impacto:

- **Procesos en estado R (Running)** Compiten por la CPU. Un número elevado indica alta carga de procesamiento.
- **Procesos en estado D (Uninterruptible sleep)** Están esperando operaciones de E/S. Su acumulación suele indicar problemas en disco o red.
- **Procesos en estado S (Sleeping)** No consumen CPU de forma activa. Su presencia es normal y no implica carga.

9.4.2 Estados de los procesos (III)

Una de las métricas clave para interpretar esta situación es el **load average**, que representa el número medio de procesos en estado **R (ejecución)** y **D (espera de E/S)** durante un intervalo de tiempo.

La interpretación del load average depende del número de núcleos del sistema:

- Valor ≈ número de núcleos → sistema en carga óptima
- Valor > núcleos → procesos en espera → posible saturación
- Valor < núcleos → capacidad disponible

El load average no mide uso de CPU, sino **nivel de carga del sistema basado en procesos activos o en espera**, siendo un indicador clave para detectar sobrecarga.

Aclaración: Procesos yes y carga del sistema

El comando yes genera un proceso que ejecuta un **bucle infinito**, consumiendo CPU de forma continua. En un sistema con **4 núcleos**, cada proceso puede ocupar un núcleo al 100%.

- 1–4 procesos → ocupación directa de núcleos (CPU → 100%)
- **>4 procesos → no hay más núcleos disponibles → los procesos se alternan en CPU**

En este caso:

- todos los núcleos siguen al **100%**
- pero hay procesos que **no pueden ejecutarse inmediatamente**
- se genera una **cola de ejecución** → aumenta el **load average**

La prioridad (nice) modifica el reparto:

- mayor prioridad → más tiempo de CPU frente a otros procesos
- menor prioridad → menos tiempo de CPU cuando hay competencia

La CPU puede estar al 100% sin bloqueo:
el sistema sigue funcionando, pero con más procesos esperando turno

9.4.3 Identificación de problemas de rendimiento

Los problemas de rendimiento se manifiestan a través de patrones observables en las métricas del sistema. La identificación de estos problemas requiere reconocer combinaciones de indicadores que reflejan limitaciones en distintos recursos.

Patrones habituales:

- **CPU saturada (CPU-bound → limitado por CPU)**
  CPU al 100% y procesos en estado **R** → uso intensivo de CPU
  Se convierte en problema cuando el **load average supera el número de núcleos** de forma sostenida, indicando que hay procesos esperando CPU
- **Presión de memoria**
  uso intensivo de **swap** → degradación del rendimiento
- **Cuello de botella en almacenamiento (I/O-bound → limitado por disco/E/S)**
  procesos en estado **D** → esperas de entrada/salida (E/S)

9.4.4 Reglas básicas para el análisis del rendimiento

- **No interpretar una métrica de forma aislada** → el rendimiento depende de la combinación de varios indicadores
- **CPU al 100% no implica necesariamente un problema** → analizar el **load average** y la existencia de procesos en espera
- **Un sistema lento no siempre está limitado por la CPU** → puede estar condicionado por memoria, disco u otros recursos
- **Identificar si los procesos están ejecutándose o esperando** → estados **R** (ejecución) vs **D** (espera de E/S)
- **Diferenciar entre carga y saturación** → carga alta ≠ degradación si no hay espera de recursos
- **Analizar tendencias, no valores puntuales** → picos aislados no implican necesariamente un problema
- **El uso de memoria no implica problema por sí mismo** → el uso de **swap** indica presión de memoria y posible degradación
- **Interpretar las métricas respecto a un baseline** → los valores deben compararse con el comportamiento habitual del sistema

9.5 Análisis del sistema mediante simulación y optimización



El análisis del rendimiento de un sistema no se limita a la observación de su comportamiento en condiciones normales, sino que requiere someterlo a situaciones de carga controlada para evaluar sus límites y su capacidad de respuesta. La simulación permite reproducir escenarios de alta demanda, mientras que la optimización implica aplicar medidas para mejorar la gestión de los recursos y mantener el funcionamiento del sistema dentro de parámetros adecuados.

9.5.1 Concepto de simulación de carga

Simulación de carga

La simulación de carga es una técnica que consiste en **generar actividad artificial** en un sistema con el objetivo de reproducir **condiciones reales o extremas de uso**. Esto permite analizar su comportamiento sin depender de la presencia de usuarios reales

¿Qué se simula?

- Uso intensivo de CPU
- Consumo de memoria
- Operaciones de disco (lectura/escritura)
- Accesos concurrentes al sistema

Objetivo

- Evaluar: **Rendimiento, estabilidad y capacidad de respuesta**
- Detectar: Cuellos de botella, saturaciones y fallos bajo carga

9.5.2 Herramientas de simulación de carga (I)

stress-ng

Herramienta de simulación de carga que permite generar consumo controlado sobre distintos recursos del sistema, como CPU, memoria, disco o procesos. Se utiliza para evaluar el comportamiento del sistema bajo condiciones de estrés y comprobar su estabilidad.

Permite definir el tipo de carga, el número de procesos implicados y la duración de la prueba, lo que facilita la reproducción de escenarios de carga de forma precisa y controlada.

Ejemplos de uso

stress-ng --cpu 4 --timeout 60s
→ Genera carga en 4 núcleos durante 60 segundos (lanza 4 workers de CPU; cada uno intenta usar el 100% de un núcleo)

stress-ng --vm 2 --vm-bytes 512M --timeout 60s
→ Genera consumo de memoria (2 procesos de 512 MB durante 60 s)

9.5.2 Herramientas de simulación de carga (II)

Otros tipos de carga

Además de CPU y memoria, **stress-ng** permite generar carga sobre otros subsistemas del sistema, lo que facilita un análisis más completo del rendimiento.

Ejemplos de uso

stress-ng --hdd 2 --timeout 60s

→ Genera carga de disco mediante operaciones de lectura/escritura (2 procesos)

stress-ng --io 2 --timeout 60s

→ Genera carga de entrada/salida a nivel general (I/O)

stress-ng --fork 4 --timeout 60s

→ Genera carga mediante creación continua de procesos

9.5.3 Herramientas de evaluación del rendimiento

sysbench

Herramienta de benchmarking utilizada para evaluar el rendimiento del sistema mediante pruebas estructuradas. Permite medir el comportamiento de distintos recursos, especialmente CPU y memoria, mediante cargas reproducibles.

A diferencia de otras herramientas, se orienta a la **medición del rendimiento** más que a la simple generación de carga, proporcionando resultados comparables.

Ejemplos de uso

sysbench cpu run

→ Ejecuta una prueba de CPU basada en cálculo intensivo

sysbench memory run

→ Ejecuta una prueba de rendimiento de memoria

9.5.4 Ejercicio: simulación de carga y optimización de CPU

Paso 1: medición del rendimiento sin carga (baseline)

Se ejecutará un proceso de prueba para medir el rendimiento del sistema en condiciones normales, sin carga adicional. Este valor servirá como referencia (baseline) para comparar el comportamiento del sistema en los siguientes pasos.

Ejecución

```
time dd if=/dev/zero of=/dev/null bs=1M count=5000
```

- Ejecutar el comando sin otros procesos activos
- Anotar el tiempo de ejecución (**real**)
- Este valor será la referencia para comparar resultados posteriores

¿Qué hace este comando?

- time mide el tiempo de ejecución del proceso
- dd copia datos desde un origen a un destino
- if=/dev/zero → genera datos (flujo continuo de ceros)
- of=/dev/null → descarta los datos (no se almacenan)
- bs=1M → tamaño de bloque (1 MB por operación)
- count=5000 → número de bloques (≈ 5 GB en total)

→ El proceso realiza un trabajo intensivo de CPU durante un tiempo limitado

9.5.4 Ejercicio: simulación de carga y optimización de CPU

Paso 2: generación de carga en el sistema

En este paso se generará carga artificial sobre la CPU mediante la ejecución de varios procesos en segundo plano. El objetivo es observar cómo aumenta el uso de CPU y cómo afecta al rendimiento del sistema.

Ejecución

```
yes > /dev/null &
```

- Ejecutar el comando varias veces (3–6 procesos)
- Cada proceso se ejecuta en segundo plano (&)
- Observar el uso de CPU con top o htop

¿Qué hace este comando?

- yes genera una salida continua de texto
- > /dev/null descarta la salida (evita saturar la terminal)
- El proceso entra en un **bucle infinito de cálculo**
- Consume CPU de forma constante

→ Cada proceso intenta ocupar tiempo de CPU de forma continua

9.5.4 Ejercicio: simulación de carga y optimización de CPU

Paso 3: medición del rendimiento bajo carga

Se ejecutará nuevamente el proceso de prueba mientras el sistema se encuentra bajo carga de CPU. El objetivo es observar cómo se ve afectado el tiempo de ejecución en comparación con el baseline.

Ejecución

```
time dd if=/dev/zero of=/dev/null bs=1M count=5000
```

- Ejecutar el comando con varios procesos yes activos
- Anotar el tiempo de ejecución (**real**)
- Comparar con el valor obtenido en el Paso 1

Análisis

- El proceso compite por el uso de CPU
- Aumenta el tiempo de ejecución debido a la planificación de procesos
- El sistema reparte el tiempo de CPU entre todos los procesos activos

→ Se evidencia la degradación del rendimiento bajo carga

9.5.4 Ejercicio: simulación de carga y optimización de CPU

Paso 4: ajuste de prioridad de procesos (optimización)

Se modificará la prioridad de los procesos que generan carga para mejorar la capacidad de respuesta del sistema. El objetivo es observar cómo influye la planificación del sistema operativo en el rendimiento.

Ejecución

- Abrir htop
- Localizar los procesos yes
- Reducir su prioridad (aumentar valor **NI**, por ejemplo +10 o +15)

¿Qué implica este ajuste?

- Los procesos yes pasan a tener **menor prioridad**
- El sistema les asigna **menos tiempo de CPU**
- Otros procesos pueden ejecutarse con mayor fluidez

→ Se optimiza el reparto de recursos sin eliminar la carga

9.5.4 Ejercicio: simulación de carga y optimización de CPU

Paso 5: nueva medición tras la optimización

Se ejecutará nuevamente el proceso de prueba tras haber ajustado la prioridad de los procesos que generan carga. El objetivo es comprobar si se produce una mejora en el rendimiento.

Ejecución

```
time dd if=/dev/zero of=/dev/null bs=1M count=5000
```

- Ejecutar el comando con los procesos yes activos pero con prioridad reducida
- Anotar el tiempo de ejecución (**real**)
- Comparar con los resultados de los pasos anteriores

Análisis

- El proceso de prueba recibe mayor tiempo de CPU
- Disminuye el tiempo de ejecución respecto al Paso 3
- Se mejora la respuesta del sistema sin eliminar la carga

→ Se evidencia el efecto de la planificación y la prioridad de procesos

9.5.4 Ejercicio: simulación de carga y optimización de CPU

Registro y análisis de resultados

Escenario	Nº procesos yes	Prioridad (NI)	Tiempo real (s)	Observaciones
Sin carga (baseline)	0	—		
Con carga		0		
Con carga (prioridad reducida)		+10 / +15		

Interpretación

- Comparar los tiempos obtenidos en cada escenario
- Analizar la influencia de la carga sobre el rendimiento
- Evaluar el impacto del ajuste de prioridad

→ Relacionar los resultados con la planificación del sistema operativo

9.5.5 Documentación y gestión de incidencias

Plantilla de registro de incidencia

Campo	Descripción
Descripción del problema	Lentitud, bloqueo parcial, degradación del sistema...
Condiciones detectadas y métricas observadas	Nº de procesos, carga de CPU, uso de memoria...
Pruebas realizadas	Comparación de baseline con situación anómala. (para esto debemos tener registradas unas métricas previas→Baseline) (time dd...)
Acciones aplicadas	Ajuste de prioridad (nice), cierre de procesos...
Resultados obtenidos	Mejora, sin cambios, empeoramiento...
Conclusión	Causa probable y valoración final